KHAN INSTITUTE OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

COMPUTER SCIENCE DEPARTMENT

Advanced Visual Programming (Theory)

BSCS VI _ A

Submitted by:

Jamila Ansar

Reg# No.

232201013

Submitted to:

Sir Uzair Hassan

Assignment No.1

March,29,2026

AI-Based Application Development

1. Introduction to AI-Based Applications

What is Artificial Intelligence?

Artificial Intelligence (AI) is a branch of computer science focused on creating systems that can perform tasks requiring human-like intelligence, such as reasoning, learning, problem-solving, and language understanding.

What are AI-Based Applications?

AI-based applications are software solutions that use AI techniques to deliver intelligent functionality. Unlike traditional apps that follow fixed rules, AI apps learn from data and adapt over time. They are adaptive, predictive, automated, and personalized.

Real-World Examples:

- Chatbots — ChatGPT, Siri (uses NLP)
- Recommendation Systems — Netflix, Amazon (uses Collaborative Filtering)
- Image Recognition — Google Photos, Face ID (uses Deep Learning)
- Fraud Detection — Banking apps, PayPal (uses Anomaly Detection)
- Virtual Assistants — Alexa, Google Assistant (uses Speech Recognition)

2. AI in the .NET Ecosystem

How AI is Integrated in .NET:

.NET supports AI through native libraries (ML.NET), REST API calls to external services (OpenAI, Azure AI), and official cloud SDKs. Developers can build intelligent apps using familiar C# syntax.

ML.NET:

ML.NET is Microsoft's open-source machine learning framework for .NET. It allows developers to train and deploy custom ML models directly in C# without needing Python. It supports tasks like classification, regression, clustering, and anomaly detection. Key tools include AutoML and the Visual Studio Model Builder.

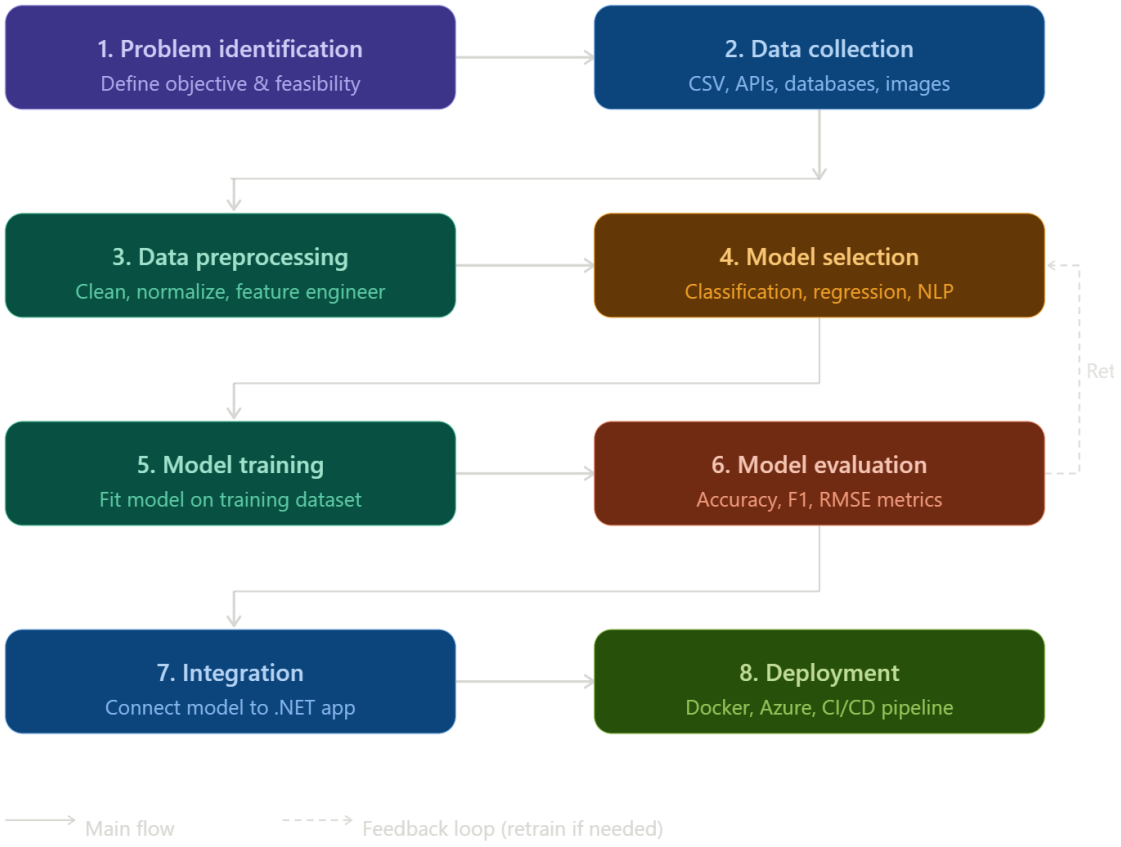
REST APIs for AI (OpenAI):

.NET apps can call the OpenAI API to access GPT models for text generation, summarization, and translation. Integration involves installing a NuGet package, storing the API key securely, and calling the API using async/await.

Benefits of .NET for AI:

- High performance with .NET 8
- Type-safe, strongly typed models
- Cross-platform (Windows, Linux, macOS)
- Rich NuGet package ecosystem
- Enterprise-ready with built-in security and logging

3. AI Application Development Lifecycle



Step	Phase	Description
1	Problem Identification	Define the business goal and AI objective
2	Data Collection	Gather structured/unstructured datasets
3	Data Preprocessing	Clean, normalize, and transform data

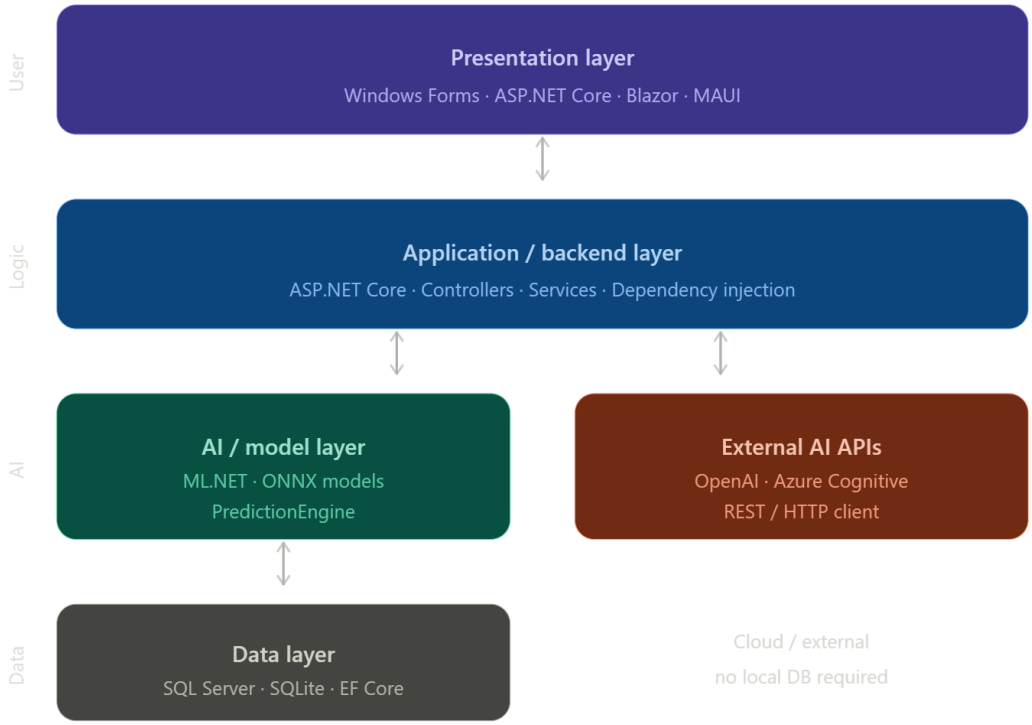
4	Model Selection	Choose the right algorithm (ML.NET, OpenAI, etc.)
5	Model Training	Train the model on the prepared dataset
6	Model Evaluation	Test accuracy, precision, and recall
7	Integration	Connect the model with the .NET application
8	Deployment	Host on cloud, Docker, or on-premise

Brief explanation of each phase:

1. **Problem Identification** : Clearly define what the AI should solve. Identify data availability and feasibility.
2. **Data Collection** : Gather data from databases, APIs, or third-party sources. Data can be structured (CSV, SQL) or unstructured (images, text).
3. **Data Preprocessing** : Handle missing values, normalize numeric data, engineer features, and split data into training/testing sets.
4. **Model Selection** : Choose algorithm based on problem type. Classification uses Random Forest or SVM; Regression uses Linear Regression; NLP tasks use Transformer models.
5. **Model Training** : Feed preprocessed data into the algorithm. In ML.NET, build a pipeline and call the Fit() method.
6. **Model Evaluation** : Test on unseen data. Key metrics: Accuracy, Precision, Recall, F1-Score for classification; MAE and RMSE for regression.
7. **Integration** : Save the model as a .zip (ML.NET) or call it via API. Connect it to the .NET backend through a service class.
8. **Deployment** : Deploy on Azure App Service, Docker containers, or on-premise servers. Use GitHub Actions for CI/CD automation.

4. Architecture of an AI-Based Application

An AI application follows a layered architecture:



Layer 1 Presentation Layer (Frontend) The user-facing part built with Windows Forms, ASP.NET Core, or Blazor. It takes user input and displays AI-generated results.

Layer 2 Application / Backend Layer Built with ASP.NET Core. Contains business logic, controllers, and service classes that coordinate communication between the UI and the AI model.

Layer 3 AI / Model Layer The intelligence of the application. Can be an ML.NET model loaded from disk, an ONNX model, or an external AI API like OpenAI or Azure Cognitive Services.

Layer 4 Data Layer (Optional) Stores user interactions, AI responses, and datasets for retraining. Uses Entity Framework Core with SQL Server or SQLite.

5. API Integration Concept

What is an API?

An API (Application Programming Interface) defines how two software systems communicate. It specifies the methods, data formats, and protocols for requests and responses.

How AI APIs are used in .NET:

AI APIs are called over HTTPS using HttpClient or SDK packages. The client sends a JSON request with the prompt and API key, and receives a JSON response with the AI-generated result.

Request/Response Flow:

Figure 3: AI API Request/Response Flow

Client App	1. User triggers action (e.g. sends a message to chatbot)
HTTP Request	2. POST /v1/messages with JSON body + API Key Header
AI API Server	3. Processes request, runs inference on model
HTTP Response	4. Returns JSON with result (text, prediction, label)
Client App	5. Parses response and displays output to user

1. User triggers an action in the app
2. App sends HTTP POST request with JSON body and API key
3. AI server processes the request and runs inference
4. Server returns JSON response with the result
5. App parses the response and displays output to the user

Key considerations: Handle rate limits with retry logic, never expose API keys in code, use async/await for all API calls, and cache frequent responses to reduce cost

6. Introduction to Docker

What is Docker?

Docker is an open-source platform that packages applications and their dependencies into lightweight, portable units called containers. Containers share the host OS kernel, making them fast and resource-efficient.

Key Concepts:

- **Image** — A read-only blueprint for creating containers
- **Container** — A running instance of an image
- **Dockerfile** — A script with instructions to build an image
- **Docker Hub** — A public registry for sharing images

Why Docker is Used:

Docker solves the "works on my machine" problem by packaging the app with its entire runtime environment, ensuring identical behaviour across all environments.

Benefits:

Aspect	Without Docker	With Docker
Environment	Machine-specific	Runs anywhere
Setup Time	Hours of config	Minutes
Portability	OS-dependent	Platform-independent
Scaling	Manual server setup	Easy horizontal scaling

7. Version Control Using GitHub

Importance of GitHub:

GitHub is the leading platform for hosting Git repositories. It enables teams to collaborate, track changes, manage issues, and automate workflows. For AI projects, it is essential because models, datasets, and code all evolve over time.

Basic Workflow:

1. Create a repository on GitHub
2. Clone it locally: git clone <url>
3. Create a branch: git checkout -b feature/ai-feature
4. Make changes and stage them: git add .
5. Commit: git commit -m "Add AI integration"
6. Push to GitHub: git push origin feature/ai-feature
7. Open a Pull Request and merge to main

README File:

Every repository should include a README.md with the project title, description, setup instructions, and usage examples. It is the front page of your project on GitHub.

8. Ethical Considerations in AI

Bias in AI: AI bias occurs when an algorithm produces unfair results due to flawed training data or model design. Types include:

- **Historical Bias** — Training data reflects past inequalities
- **Sampling Bias** — Dataset does not represent the full population
- **Algorithmic Bias** — The model amplifies certain patterns

Mitigation: Audit datasets for diversity, test across demographic groups, and include diverse development teams.

Data Privacy: AI systems often require large amounts of personal data, creating privacy risks. Key regulations include GDPR (EU), CCPA (California), and HIPAA (healthcare). Best practices: collect only necessary data, anonymize PII, encrypt data, and always obtain user consent.

Responsible AI Usage: Core principles:

- **Fairness** — Treat all users equitably
- **Transparency** — Users should understand AI decisions
- **Accountability** — Developers are responsible for AI behaviour
- **Safety** — Thoroughly test before deployment
- **Human Oversight** — Critical decisions must involve human review

Conclusion

This assignment covered the complete landscape of AI-based application development in .NET from foundational AI concepts and the development lifecycle to architecture design, API integration, Docker, GitHub, and ethical considerations. Developers working with .NET are well-positioned to build powerful, responsible AI applications using tools like ML.NET, OpenAI APIs, Docker, and GitHub.

References:

- OpenAI API Docs: platform.openai.com/docs
- Docker Docs: docs.docker.com
- GitHub Docs: docs.github.com